

COMMON COURSE ASSIGNMENT FORMAT

A. Assignment Information

Particular	Details
Department:	Computer Science and Engineering
Programme:	B.E./B.Tech. Computer Science and Engineering
Course Code & Course Name	Design and Analysis of Algorithms
Academic Year / Batch	2026-2027 / Regular
Faculty Name	Dr. F. Mary Harin Fernandez
Assignment Title	Development of an Efficient Hybrid Algorithmic Solution for Large-Scale Geospatial Data Processing
Date of Issue	31 Aug 2026
Date of Submission	1 September 2026
Maximum Marks	100
Course Outcome(s) – CO	CO1: Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. CO2: Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems CO3: Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication
Bloom's Taxonomy Level	L4 – Analyze, L5 – Evaluate, L6 – Create
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Efficient processing of large-scale real-world data with reduced computational time and resource utilization

TEAM MEMBERS :

Lanka Kavya(192472298)

Sathwika Reddy(192525274)

Mridhula(192565070)

1. Overview & Choice of Algorithmic Solution

Algorithms Evaluated

1. **Brute-Force Closest Pair:** Evaluates all $n(n-1)/2$ possible point pairs exhaustively to find the pair with the minimum Euclidean distance.
2. **Divide-and-Conquer Closest Pair:** Recursively splits the point set about the median x-coordinate, solves the subproblems for left and right halves, and checks a central dividing strip of width 2δ .
3. **Adaptive Hybrid Closest Pair (Selected Solution):** Integrates Divide-and-Conquer and Brute-Force by recursively partitioning the dataset until the subproblem size reaches an experimentally determined threshold (S_0). Below this threshold, it dynamically switches to the Brute-Force approach.

Justification: Why the Adaptive Hybrid Approach Was Chosen

While Brute-Force is simple and Divide-and-Conquer provides optimal asymptotic bounds, neither represents the most efficient *practical* implementation on real-world geospatial data:

- **Limitation of Standalone Brute-Force:** As dataset size grows ($n=10^3, 10^4, 10^5$), its $O(n^2)$ time complexity creates severe execution bottlenecks, making it computationally infeasible for large-scale infrastructure planning.
- **Limitation of Standalone Divide-and-Conquer:** Although its asymptotic time complexity is $\Theta(n \log n)$, recursive partitioning introduces substantial non-computational overhead. For very small subproblems ($n \leq 15$ or 30), setting up recursion stack frames, making function calls, computing medians, and creating sub-arrays takes significantly more execution time than simply running a quick $O(n^2)$ nested loop.

- **Superiority of the Adaptive Hybrid Solution:** The hybrid algorithm combines the macro-scalability of Divide-and-Conquer with the micro-efficiency of Brute-Force. By cutting off recursion early at threshold S_0 , it eliminates high call-stack and array-slicing overhead for small input sizes while preserving the optimal $O(n \log n)$ asymptotic theoretical growth for large datasets.

2. Problem Understanding and Formulation

Problem Context & Real-World Infrastructure Planning

Modern geospatial infrastructure systems—such as telecommunication networks, weather monitoring setups, and air traffic control systems—rely heavily on precise coordinate processing. In high-density facility mapping, identifying the closest pair of locations is critical for detecting redundant infrastructure, avoiding signal or physical interference between neighboring stations, and optimizing coverage across regions.

Input & Output Definitions

- **Input:** A dataset $P=\{p_1, p_2, \dots, p_n\}$ containing n 2D spatial coordinate pairs, where each point $p_i=(x_i, y_i)$ represents a facility's geographical position.
- **Output:** The pair of points (p_i, p_j) where $i \neq j$ that minimizes the Euclidean distance $d(p_i, p_j)$, alongside the calculated minimum distance value δ .

Distance Metric Formulation

The spatial distance between any two facilities $p_i=(x_i, y_i)$ and $p_j=(x_j, y_j)$ is computed using the standard 2D Euclidean distance formula:

$$d(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

Computational Challenges & Scalability Constraints

- **$O(n^2)$ Complexity Bottleneck:** The naive brute-force approach compares every unique point pair. For a dataset with n records, the total number of distance operations is:

$$\frac{n(n-1)}{2}$$

As n grows to 10^5 or 10^6 coordinates, n^2 grows quadratically (e.g., 10^{10} to 10^{12} comparisons), leading to execution bottlenecks, high CPU utilization, and latency issues.

- **Memory & Processing Limits:** Performing global sorting and maintaining nested pairwise loops on dynamic or large datasets exhausts memory and leads to excessive processor thrashing.

Key Assumptions & Constraints

1. Coordinates are mapped to a 2D Cartesian plane using planar projections.
2. Dataset records are free of duplicate coordinate entries that yield a trivial zero distance.
3. Computational evaluations are conducted in identical runtime environments across all three algorithmic variants to ensure consistency.

3. Theoretical Complexity Analysis

1. Brute-Force Closest Pair

- **Mechanism:** Checks all $n(n-1)/2$ possible combinations.
- **Time Complexity Analysis:**
 - **Worst Case:** $\Theta(n^2)$
 - **Best Case:** $\Omega(n^2)$ (always checks all pairs regardless of point layout)
 - **Average Case:** $O(n^2)$
- **Space Complexity:** $O(1)$ auxiliary space (excluding input array storage).

2. Divide-and-Conquer Closest Pair

- **Mechanism:** Sorts points by x-coordinate, splits the dataset into left and right halves around median line x_{mid} , recursively finds minimum distances δ_L and δ_R , sets $\delta = \min(\delta_L, \delta_R)$, and checks a central strip of width 2δ sorted by y-coordinate.
- **Time Complexity Analysis:**
 - Presorting takes $O(n \log n)$ time.
 - Strip processing checks at most 7–8 points for each point inside the strip, bounded by $O(n)$.
 - Total recurrence: $T(n) = 2T(n/2) + O(n)$.
 - **Overall Time Complexity:** $\Theta(n \log n)$.
- **Space Complexity:** $O(n)$ space required for storing sorted coordinate arrays and strip structures.

3. Adaptive Hybrid Closest Pair

- **Mechanism:** Recursively splits subproblems using Divide-and-Conquer until subproblem size $n \leq S_0$ (where S_0 is the threshold size), at which point it switches to Brute-Force.
- **Time Complexity Analysis:**
 - Asymptotic behavior remains $O(n \log n)$ because S_0 is a fixed constant.
 - Reduces recursive overhead, stack frame creations, and array copying overheads for small inputs.
- **Space Complexity:** $O(n)$ for dynamic strip handling and recursion stack.

4. Step-by-Step Master Theorem Recurrence Proof

To analyze the Divide-and-Conquer recurrence relation:

$$T(n) = 2T\left(\frac{n}{2}\right) + f(n), \quad \text{where } f(n) = O(n)$$

Step 1: Identify Master Theorem Parameters

The general Master Theorem recurrence equation is:

$$T(n)=aT\left(\frac{n}{b}\right)+f(n)$$

From our divide-and-conquer function:

- $a=2$ (number of subproblems generated per recursive call)
- $b=2$ (factor by which input size is divided)
- $f(n)=O(n)$ (work done outside recursion for dividing and merging strip points)

Step 2: Compute $n^{\log_b a}$

$$n^{\log_b a}=n^{\log_2 2}=n^1=n$$

Step 3: Compare $f(n)$ with $n^{\log_b a}$

- $f(n)=\Theta(n)$
- $n^{\log_b a}=\Theta(n)$

Since $f(n)=\Theta(n^{\log_b a})$, Case 2 of the Master Theorem applies.

Step 4: Apply Case 2 Formula

Under Case 2 of the Master Theorem:

$$T(n)=\Theta(n^{\log_b a} \cdot \log n)$$

Substituting $n^{\log_2 2}=n$:

$$T(n)=\Theta(n \log n)$$

Conclusion: The theoretical time complexity of the Divide-and-Conquer core is strictly $\Theta(n \log n)$.

5. Hybrid Switching Strategy & Architectural Flowchart

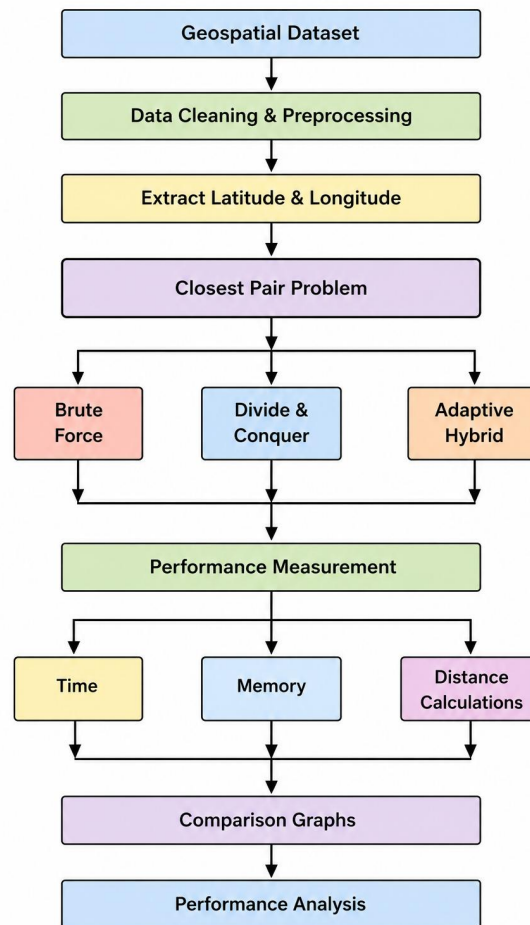
Mathematical Justification of Hybrid Switching

While Divide-and-Conquer exhibits asymptotic superiority ($\Theta(n \log n)$ vs. $\Theta(n^2)$), the constant factors hidden within the Big-O notation differ significantly:

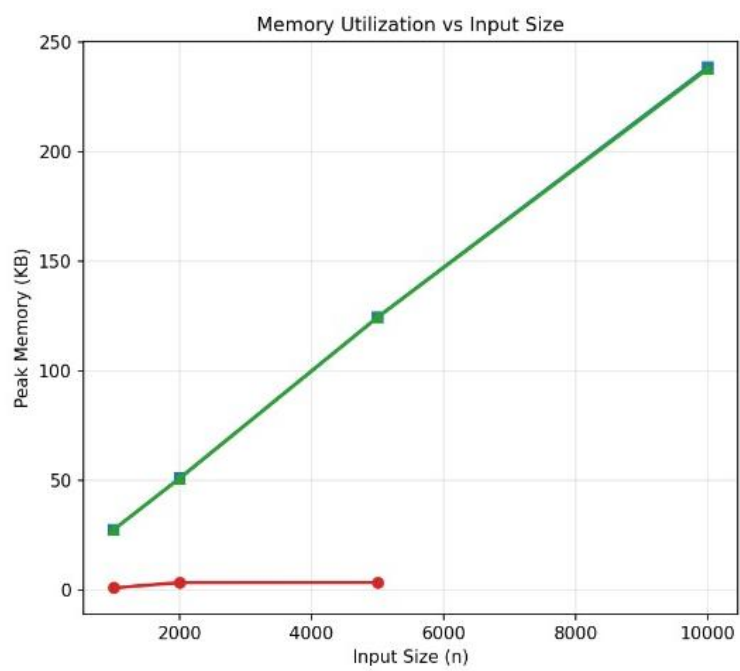
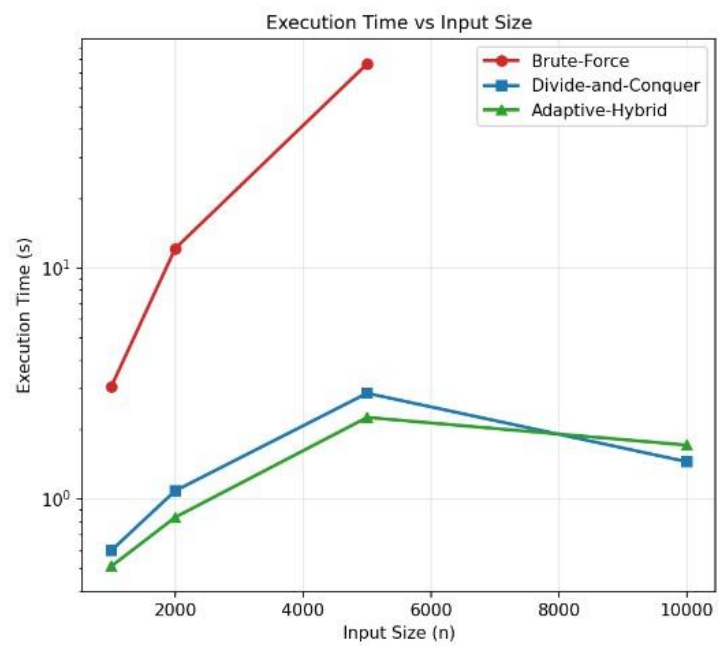
- **Divide-and-Conquer Running Time:** $C_{D\&C} \cdot n \log_2 n + \text{Recursion Overhead}$
- **Brute-Force Running Time:** $C_{BF} \cdot \frac{n(n-1)}{2}$

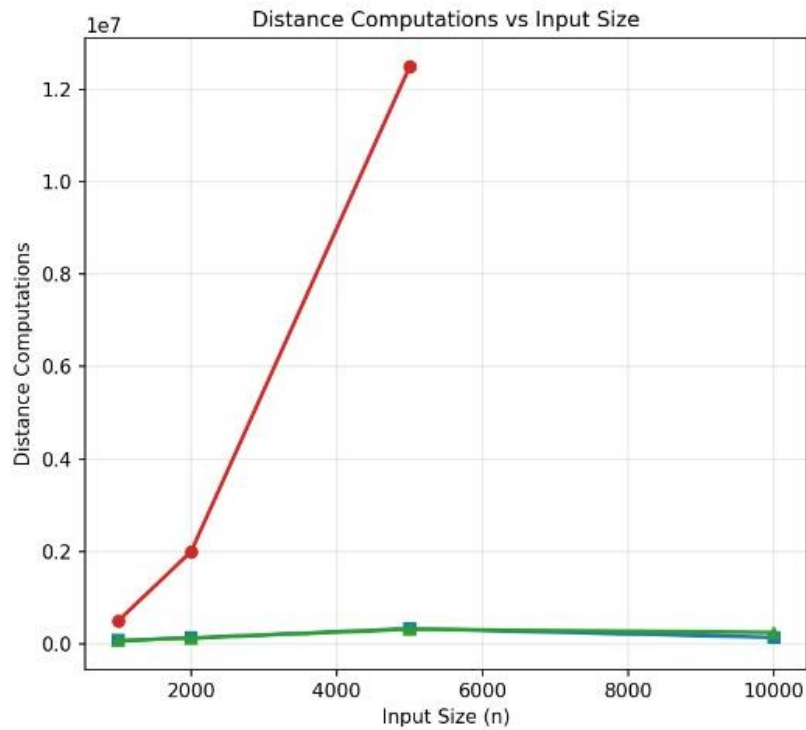
For small subproblems ($n < S_0$), $C_{D\&C} > C_{BF}$ due to system stack operations. By introducing a dynamic cutoff point S_0 , the algorithm bypasses recursion overhead for small subsets without increasing total asymptotic complexity.

Architecture Diagram:



Comparison Graphs:





6. Threshold & benchmark performance analysis :

Final Performance Comparison :

Input Size	Brute Force Time (s)	D&C Time (s)	Hybrid Time (s)	BF Distances	D&C Distances	Hybrid Distances
100	0.041965	0.002921	0.002973	4,950	110	122
500	0.945402	0.020931	0.019579	124,750	532	837
1,000	4.183207	0.046656	0.047578	499,500	1,081	1,698
2,000	16.972906	0.099826	0.093704	1,999,000	2,175	3,405

Input Size	Brute Force Time (s)	D&C Time (s)	Hybrid Time (s)	BF Distances	D&C Distances	Hybrid Distances
5,000	104.588459	0.292670	0.331480	12,497,500	6,123	10,663

Execution Time Analysis :

Brute Force execution time increased rapidly from **0.041965 s** at 100 points to **104.588459 s** at 5,000 points. D&C required only **0.292670 s**, while Hybrid required **0.331480 s** at 5,000 points. Thus, D&C and Hybrid scale much better than Brute Force.

Threshold Optimization:

Eight threshold values from **5 to 50** were tested. The lowest average execution time was **0.260426 s** at **threshold 5**. Therefore, **5 was selected as the optimal Hybrid threshold**.

Threshold Analysis:

A threshold of 5 resulted in only **10,663 distance calculations**, compared with **46,424** at threshold 20 and **95,172** at threshold 40. Larger thresholds increase Brute-Force processing, resulting in more distance calculations and higher execution time.

Theoretical vs Experimental Comparison:

Algorithm	Theoretical Time	Experimental Performance
Brute Force	$O(n^2)$	Increased rapidly with input size
Divide & Conquer	$O(n \log n)$	Very good scalability
Hybrid	$O(n \log n)$	Very good scalability with threshold 5

Engineering Decision:

Brute Force is suitable for small datasets but does not scale well. D&C provided the fastest performance for the tested dataset, while Hybrid with **threshold 5** provided an efficient adaptive solution by combining D&C with Brute Force. Therefore, **D&C is the best-performing algorithm in the experiment, while Hybrid is the preferred adaptive approach.**

B. Assessment Rubric

Criteria	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & Algorithmic Formulation	10	Clearly identifies the problem, requirements, inputs, outputs, constraints and computational challenges and formulates them appropriately for solution development.	Problem and major requirements are correctly formulated with minor gaps.	Basic formulation is provided, but requirements or constraints are partially identified.	Problem is poorly understood or inadequately formulated.
Algorithmic Solution Design	15	Designs a clear, logical and feasible solution strategy with appropriate algorithmic techniques, pseudocode/flowchart and well-justified design decisions.	Appropriate solution design with minor gaps in logic, representation or justification.	Basic solution design is presented but lacks completeness or clear justification.	Solution design is inappropriate, incomplete or unclear.
Development	20	Develops a meaningful	Develops an	Solution	Merely

/ Adaptation / Optimization of Algorithmic Solution		algorithmic solution through integration, adaptation, modification, hybridization or optimization of suitable techniques; demonstrates clear improvement or suitability for the identified problem.	appropriate solution with meaningful adaptation or improvement , with minor limitations.	demonstrates limited adaptation or development beyond standard implementation.	reproduces an existing algorithm with little or no meaningful development or adaptation.
Implementation & Modern Tool Usage	15	Developed solution is correctly and efficiently implemented using appropriate programming/computational tools and handles relevant input conditions reliably.	Implementation is functional with minor logical or efficiency issues.	Core implementation works but contains limitations or incomplete functionality.	Implementation is incomplete, unreliable or non-functional.
Efficiency, Testing & Validation	15	Performs appropriate complexity evaluation and systematic testing using suitable data/input sizes; validates correctness, efficiency and scalability with clear evidence.	Complexity evaluation and testing are mostly appropriate with minor gaps.	Basic testing and efficiency evaluation are provided but lack sufficient validation.	Testing, complexity evaluation or validation is inadequate or substantially incorrect.
Performance Improvement , Comparison & Final	15	Demonstrates the effectiveness of the developed solution through appropriate	Good performance comparison and	Basic comparison is provided, but improvement	Improvement is not demonstrated or the final

Decision		comparison; critically evaluates performance, limitations and trade-offs and justifies the final algorithmic decision with evidence.	justification with minor limitations.	or final justification lacks depth.	algorithmic decision is unsupported.
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	10	Thoughtful justification of algorithmic design and development decisions; clear connection to relevant SDG(s); specific reflection on computational challenges, performance improvements, trade-offs, and learning gained.	Appropriate justification of algorithmic decisions; SDG relevance, challenges, improvements, and learning are discussed but lack depth.	Reflection is present but generic; limited justification of algorithmic decisions, improvements, SDG relevance, or learning outcomes.	No genuine reflection is provided, or the reflection lacks meaningful connection to algorithm development, SDG relevance, challenges, or learning outcomes.
Total	100				

B. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem Understanding & Algorithmic Formulation	CO1	PO1, PO2	L4	10
Algorithmic Solution Design	CO2, CO3	PO2, PO3	L4, L6	15

Development / Adaptation / Optimization of Algorithmic Solution	CO2, CO3	PO2, PO3	L5, L6	20
Implementation & Modern Tool Usage	CO2, CO3	PO3, PO5	L3, L6	15
Efficiency, Testing & Validation	CO1, CO2, CO3	PO2, PO4	L4, L5	15
Performance Improvement, Comparison & Final Decision	CO1, CO2, CO3	PO2, PO4	L4, L5	15
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	CO1, CO2, CO3	PO7, PO10, PO12	L4, L5	10

C. Faculty Evaluation & Continuous Improvement CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action: